Project 1: Business Management System

CONSOLE is growing fast, but our current way of working is lagging behind. Using messy spreadsheets and scattered chat messages for a firm with 150+ developers is no longer enough. We need a "Centralized Source of Truth"—one platform where everything happens. This project is about building that professional system to stop the "digital mess" and make sure everyone is accountable.

Goal:

Your goal is to develop a **web application** that contains features which current business management platforms lack. Try to make it general so that various firms can use it. For reference you can explore platforms like [Jira | Issue & Project Tracking Software | Atlassian].

The 4-Stage Approach:

Stage 1: The Foundation

Goal: Make a functional website.

1. **Smart Login System:**
    - Build a secure login where the software knows your role.
    - Every single person should see completely different dashboards based on their role.
    - Face recognition for login and authorization purposes. Look up *Open-CV.*
2. **Task Assignment and Monitoring:**
    - The employer can assign the task to its employees and can monitor them.
    - The employees get their tasks in their tasks list and can update tasks status:
        i. To-Do → In-Progress → Completed
    - *Search the term **"Kanban"** online, you might find something interesting*
3. **The "Black Box" Recorder:**
    - Create a history log that records every action.
    - If a task is deleted or changed, the system must show who did it and when, so there is no "mystery" behind missed deadlines.
4. **Clean Interface:**
    - Build a simple, easy-to-use layout that works on phones and PCs.

Stage 2: The Automator and Communicator

Goal: Let the code do the boring stuff.

1. **Google Calendar and Google Meet Integration:**
    - Tasks should automatically appear on a user's Google Calendar so they never forget a deadline.
    - Auto arranging the meeting based on availability (according to their google calendar) of all persons involved.

    - **Optional:** Instead of depending on Google Calendar, you can also make a local calendar and reminding system.
2. **Automatic Invoices:**
    - The system should automatically create PDF salary slips and bills on the 1st of every month and after confirmation automatically send it to the respective person.
    - No more relying on human memory for payments.
    - Sends mails regarding the bills, salary and imported announcement automatically. This time through Gmail Api.
3. **Local Messaging or Mailing:** A localized mailing or messaging for faster interaction and a centralized place for all messages/mails of tasks assigned or announcements for the employees.

Stage 3: The Live Nerve Center

Goal: Everything updates instantly.

1. **Live Scoreboard:**
    - Build a dashboard that updates in real-time.
    - As soon as a task is finished or a payment is made, the screen should update for everyone without needing to refresh.
2. **Active Users:**
    - Show who is currently online or working on a task, just like a "live" office.

Bonus Stage: God Mode

1. **The "Burn Rate" Graph:** Create a visual chart that predicts when the company will run out of cash based on spending. In this don't use already existing LLMs for predictions, use a prebuilt model and integrate it to the website.
   Checkout: Hugging Face to find a suitable model.
2. **Optimization:** Minimize the delays.

Project 2: Smarter BlinkIt

Most shopping apps today are just digital catalogs. If you want to "cook an Italian dinner," you have to search for every single item—pasta, sauce, cheese—one by one. It's slow and boring. On the other side, small shop owners have to manually type in every single product they sell, which leads to mistakes and "out of stock" headaches. We need a "Smart Marketplace"—a website that understands what you want to do, not just what you type.

Goal:

Your goal is to **build a web application** that connects buyers and local sellers. Instead of just a search bar, you'll build an **"AI Shopping Assistant"** that fills your cart for you and a "**Barcode based Inventory Modification System".**

The 4-Stage Approach:

Stage 1: The Foundation

Goal: Build a working website where users can find what they need.

- **Dual-Login System:**
  - Create a secure login where the app knows if you are a "Buyer" or a "Seller." Each should see a different dashboard.
  - Face recognition for login and authorization purposes. Look up *Open-CV.*
- **"Intent" Search:**
  - Instead of just matching words, try to make the search smart. If a user types "I have a cold," the website should suggest **Honey** or **Ginger Tea** automatically. *Research Tip: Look up "Semantic Search" to see how this works!*
- **Local First:**
  - The website should detect the user's location and auto place the order to the closest shops first so that delivery is fast and cheap.
- **Barcode based Inventory Modification:**
  - Sellers are busy. Build a feature where a seller can share barcodes on boxes to update their inventory with ease.
- **Dummy Payment:**
  - Buyers can complete orders using a test payment flow to simulate a secure real-world checkout experience. (Tip : "Razorpay" provides test payment integration)

Stage 2: The Automator

Goal: Let the AI do the heavy lifting.

- **The "Recipe" Agent:**

    - Imagine typing "Make Pizza for 4 people." The website should automatically find the flour, cheese, and tomato sauce of the required amount from nearby shops and put them all in your cart in one click.
- **Similar Items Suggestion:**
    - When a buyer views a product, the app should automatically show helpful suggestions. For example, if someone is viewing a pasta brand that is out of stock, the system can suggest other pasta brands. It can also show items people usually buy together, like pasta sauce or cheese.
    - ***Important Usage*** *: Use Databases that store data as Graphs (**Neo4j**) Store products as nodes and connect them with simple relationships like* `SIMILAR_TO` *and* `BOUGHT_WITH`*. Use basic Cypher queries to fetch the most connected products.*

Stage 3: The Orchestrator (Logistics & Quality)

Goal: Make sure the order is right and the route is fast.

- **Smart Cart Splitting:**
    - If Shop A has the pasta but Shop B has the cheese, the website should be smart enough to split the order and find the best way to deliver both to the user's house.
- **Live Storeboard:**
    - Create a real-time dashboard for the owner that shows which items are selling the fastest and which shops are the "Top Rated" in the city right now.

Bonus Stage: God Mode

Goal: Prediction based features

- **The "Money Map":** A visual chart that shows which neighborhoods are buying the most and where new shops should open and also which types of items are majorly sold in the neighbourhood.
- **Smart Product Pairing:** Integrate a prebuilt model which predicts which two products can be displayed together based on whether they are majorly sold together in the past. For more info search for *Beer and Diaper* case study to understand how such analyses are helpful and how to implement it using the prebuilt ML model.
  Checkout: [Hugging Face](#) to find a suitable model.

Project 3: TalentBridge

CONSOLE is scaling fastly, but our hiring process is lagging behind. Top-tier talent is slipping through the cracks because our data is scattered across external testing sites, messy email threads, and rigid search filters.

Goal:

Your goal is to develop a **web application** that contains features which recruitment systems lack.

The 4-Stage Approach:

Stage 1: The Functional Core

Goal: Make a functional website containing a secure, end-to-end recruitment pipeline.

- **The Dual-Sided Portal:**
  - Build a specialized interface for both Recruiters and Candidates. Here both can search each other by keyword and filter based searching.
- **Direct Sync Messaging:**
  - An in-built communication hub for real-time updates and for direct OA and Interview links can be shared.
- **The Audit Trail:**
  - Every status change (Applied → Interviewing → Offered) must be logged to prevent "The Concurrency Collision".

Stage 2: The Smart Matcher

Goal: Solve the "Keyword Trap" with AI-driven discovery.

- **Hybrid Search Engine:**
  - Instead of using simple searching, use an integrated Semantic Layer (RAG) that understands the *intent* of a job description. *Look up "Semantic Searching " to see how this works!*
- **Contextual Ranking:**
  - The system should rank candidates based on how their actual experience aligns with the role, even if their resume uses different phrasing than the job post.

Stage 3: The Extraordinary Features

Goal: Add few features easing the most important and time consuming tasks.

- **Automation in Messaging and Reminders in Calendar:**
  - When a recruiter assigns an Online Assessment (OA) or an interview link, the system sends automated reminders and syncs the deadline directly to the candidate's personal calendar.
- **Resume → Custom Profile:**
  - A feature which automatically converts your resume into a custom profile which will help recruiters to easily identify the best candidate.
- **Offer Letter Generator:**
  - Automatically generate secure, legal-ready PDFs for offers.
- **Auto Closure:**
  - The job position closes once the offer letter is accepted by the candidate

Bonus Stage: The Profile Vault (Integrated OA)

Goal: Eliminate third-party dependencies and "Evaluation Black-Holes."

- **The Coding Sandbox:** Develop an integrated IDE (You may integrate third party IDE provider) and Quiz Engine within the platform. Candidates take technical tests directly in their profile.
- **Live-Monitoring Layer:** Create a "Proctor" feature that tracks tab-switching and allows recruiters to watch code being written in real-time.
- **Instant Grading:** Upon submission, the system must auto-grade the test and instantly update the candidate's status in the "Live Scoreboard."
- *For reference you can explore platforms like CoderPad, CoderPad majorly focus on interviews and OA related things so for that part you can refer to it.*

Made with ♡ by <TeamCONSOLE/>